

Layered Prompt Protection for AI Agents in Customer-Controlled Cloud Accounts: Model Robustness and Defense-Layer Contribution

Protección en capas de prompts para agentes de IA en cuentas de nube controladas por el cliente: robustez frente al modelo y contribución de las capas de defensa

David Petrocelli¹  and Juan Manuel Fernández^{1,2} 

¹National University of Luján, Luján, Argentina

²Buenos Aires Institute of Technology (ITBA), Buenos Aires, Argentina
dpetrocelli@unlu.edu.ar, jmfernandez@{unlu,itba}.edu.ar

Abstract

Large language models relocate a substantial share of application logic from executable code to natural-language system prompts. When AI agents must run inside customer-controlled cloud accounts for data residency, these prompts are exposed to privileged principals despite being proprietary intellectual property. We treat this as a cross-domain trust problem and present a defense-in-depth architecture that composes four software-based layers, cryptographic prompt delivery, OWASP-aligned input validation, output filtering with steganographic watermarking, and cross-account monitoring, into a trust chain that reduces prompt exposure and supports output-integrity controls and forensic traceability under the stated trust assumptions, without Trusted Execution Environments. It is implemented with standard serverless services and validated through penetration testing and a live two-account deployment. This article extends our conference work with a multi-model evaluation of nine LLMs from six families under an identical protocol and an ablation study of the defense layers. With the full pipeline no final response met the verbatim leakage criterion for any model, while the block rate ranged from 62.7% to 93.7% across models. Input validation contributes a model-independent floor of 59.5%, prompt hardening supplies most of the remaining block rate with a strongly model-dependent effect, and output filtering removes the residual verbatim leakage.

Keywords: AI security, defense-in-depth, multi-tenant cloud, prompt protection, watermarking

Resumen

Los LLM trasladan gran parte de la lógica aplicativa del código ejecutable a *system prompts*. Cuando los agentes de IA deben ejecutarse en cuentas de nube del cliente por residencia de datos, esos prompts quedan expuestos a principales con privilegios aunque son

propiedad intelectual. Tratamos el problema como uno de confianza entre dominios y presentamos una arquitectura de defensa en profundidad de cuatro capas (entrega criptográfica del prompt, validación de entradas alineada con OWASP, filtrado de salidas con marca de agua esteganográfica y monitoreo entre cuentas) que reduce la exposición del prompt y provee controles de integridad de las salidas y trazabilidad forense bajo los supuestos de confianza declarados, sin entornos de ejecución confiables. Extendemos nuestro trabajo de conferencia con una evaluación multimodelo de nueve LLM de seis familias y un estudio de ablación de las capas. Con el pipeline completo ninguna respuesta final cumplió el criterio de fuga literal, con tasas de bloqueo entre 62,7 % y 93,7 %. La validación de entradas aporta un piso del 59,5 % independiente del modelo, el endurecimiento del prompt provee la mayor parte del bloqueo restante con un efecto muy dependiente del modelo, y el filtrado de salidas elimina la fuga literal residual.

Palabras claves: defensa en profundidad, marca de agua, nube multiinquilino, protección de prompts, seguridad de IA

1 Introduction

Over the last few years, large language models (LLMs) have changed the way software systems encode and operationalize knowledge [1, 2]. Conventional systems concentrate their intellectual value in executable code, where behavior is implemented explicitly. AI agents, by contrast, are increasingly governed by *system prompts*: natural-language specifications of behavior, domain expertise, and safety constraints that act as the primary control interface and externalize system logic into interpretable text [3, 4, 5]. A considerable share of the logic that used to reside in compiled artifacts therefore now lives in natural-language specifications, which creates new challenges for intellectual property protection whose practical relevance grows

with the *AI agent as a service* market [6].

The problem is most acute in multi-tenant cloud environments, where deployment and data ownership belong to different administrative domains. AI agents are frequently required to execute inside the customer’s own cloud account in order to satisfy data residency constraints and regulatory frameworks such as the General Data Protection Regulation (GDPR), the Health Insurance Portability and Accountability Act (HIPAA), and Service Organization Control 2 (SOC 2) [7]. At the same time, the system prompt must be treated as proprietary knowledge, because it encodes the differentiated capabilities on which the commercial value of the service rests. The two requirements conflict: running the agent in a customer-controlled environment opens an exposure surface in which prompt configurations become readable by privileged actors [8, 9]. The conflict is practical rather than hypothetical. Managed agent platforms such as Amazon Bedrock Agents, Azure AI Agent Service, and Vertex AI Agents keep agent configurations in account-level resources that administrative principals can usually read, prompt extraction has been demonstrated without any privileged access [10, 11], and prompt injection [12, 13], ranked first in the Open Worldwide Application Security Project (OWASP) LLM Top-10, enlarges the attack surface further.

Existing defenses cover this problem only partially and, as a rule, in isolation. Cryptographic mechanisms protect prompts in transit or under controlled access [14]; software defenses address individual vectors through filtering, obfuscation, or leakage detection [15, 16, 17, 18, 19]; hardware-based approaches rely on Trusted Execution Environments (TEEs), which offer strong isolation but require dedicated hardware, add measurable overhead [20], and provide no support for forensic attribution [21, 22]. Surveys of agentic security [9] report no deployed architecture that protects prompts systematically across administrative domains. We therefore frame prompt protection in multi-tenant LLM systems as a cross-domain trust problem [23, 24, 9]: how trust is split between vendor and customer environments, how adversaries exploit the resulting trust boundary, and how several defenses can be composed so that the guarantees survive the compromise of any single one.

This article presents a defense-in-depth architecture that composes standard cloud services into a four-layer trust chain: cryptographic prompt delivery (Layer 1), OWASP-aligned input validation (Layer 2), output filtering with steganographic watermarking (Layer 3), and cross-account monitoring (Layer 4). The composition reduces prompt exposure and supports output-integrity controls and forensic traceability under the stated trust assumptions, while preserving deployment flexibility; since it depends on no specialized hardware and distributes the defenses across independent layers, it is portable across cloud providers and tolerant to the

failure of individual layers. A layered design, however, raises two questions that a single deployment cannot answer. Layers 2 and 3 interact with the behavior of the underlying LLM, so their effectiveness may vary across model families, and the value of composing several layers rests on the assumption that they are complementary, which has to be shown empirically.

- **RQ1 (model robustness).** How robust is the proposed defense-in-depth architecture across different LLM families?
- **RQ2 (layer contribution).** How much does each of the defenses that act on requests and responses contribute to the security effectiveness of the architecture, and are they complementary?

This article is an extended and revised version of our work presented at JCC-BD&ET 2026 [25], which introduced the threat model, the four-layer architecture, the serverless reference implementation, and its validation through controlled penetration testing and a live two-account deployment. The present version extends that work with the following contributions: (i) a systematic multi-model evaluation that applies an identical experimental protocol to nine LLMs from six model families, both proprietary and open-weight, in order to answer RQ1; (ii) a defense-layer ablation study that quantifies the individual and complementary contribution of the defenses acting on the content path, including the cases in which an attack that passes Layer 2 is intercepted by Layer 3, while Layers 1 and 4 are validated functionally, in order to answer RQ2; and (iii) an expanded experimental methodology, a more detailed account of the reference implementation and of its reproducibility environment, and a discussion informed by the new evaluations. The sections shared with the conference version have been revised and rewritten.

Section 2 reviews related work; Sections 3 to 5 present the threat model, the architecture, and its reference implementation; Sections 6 to 8 present the methodology, the security evaluation, and the performance and cost evaluation; Section 9 discusses results and limitations, and Section 10 concludes.

2 Related Work

Prompt extraction and manipulation attacks against LLM applications have received considerable attention. PLeak [10] casts prompt extraction as an optimization problem and reconstructs system prompts almost completely from crafted queries; multi-turn leakage attacks [11] exploit the sycophancy of instruction-tuned models to recover hidden instructions incrementally, with success rates that reach 86.2%; and systematic reviews of prompt injection [12] show that keeping the prompt confidential is not sufficient to guarantee secure behavior.

Software-based defenses respond to these threats at specific points of the interaction pipeline. ProxyPrompt [15] interposes a proxy that shields the real prompt, SysVec [16] replaces the textual prompt with a non-invertible vector representation, PromptGuard [18] applies structured filtering against injection, Encrypted Prompt [17] gates prompt access through permission-bearing tokens, PromptKeeper [19] frames leakage detection as hypothesis testing, and Attention Tracker [26] inspects model internals for attention patterns typical of adversarial inputs. Each operates inside a single trust boundary and targets one attack vector at a time. A second line of work composes several defenses: TRYLOCK [27] stacks jailbreak defenses and Countermind [28] adds auditing and monitoring to the inference pipeline, while analyses of serverless machine learning security [29] describe the enlarged attack surface of cloud-native deployments; these contributions concentrate on inference-time robustness and leave cross-domain deployment unaddressed. At the infrastructure level, TEE-based systems such as Petridish [21] and Omega [22] run inference inside secure enclaves, with moderate overheads under controlled conditions [20], but depend on specific hardware and offer no forensic attribution or cross-system monitoring. Watermarking addresses the complementary problem of output provenance: token-level schemes [30], scalable frameworks such as Waterfall [31] and SynthID [32], and zero-width character (ZWC) steganography [33], whose covert channel can also be abused [34]; these techniques target attribution, not the confidentiality of system prompts.

Taken together, the literature treats prompt confidentiality, input validation, output filtering, and attribution as separate problems, usually inside one trust boundary, and proposals that combine several mechanisms rarely report how much each component contributes or whether their effectiveness holds when the underlying model changes. This article composes cryptographic protection, validation, filtering, watermarking, and monitoring across layers and administrative boundaries, and evaluates the result across model families and layer configurations.

3 Threat Model and Security Requirements

This section formalizes the threat model for multi-tenant LLM deployments, in which deployment and ownership are separated across administrative boundaries: it defines the system model, characterizes the adversary classes, and states the security requirements together with the attack vectors they must withstand.

3.1 System Model

The architecture is defined over a *two-account boundary*. The *Vendor Account* is trusted and controlled

by the Software-as-a-Service (SaaS) vendor; it hosts the prompts, the cryptographic keys, the container registry, and the monitoring infrastructure. The *Customer Account* is semi-trusted; it hosts the deployed agent, an Application Programming Interface (API) gateway, and an event-forwarding component. The boundary maps onto Amazon Web Services (AWS) account, Azure subscription or tenant, and Google Cloud Platform (GCP) project boundaries, so the model does not depend on a particular provider.

3.2 Adversary Classes

Three adversaries with increasing capabilities are considered. The **Customer Administrator** ($\mathcal{A}_1$) holds full privileges over the customer account: it can inspect deployed resources and container images, read environment variables, and intercept traffic. Its motivation ranges from curiosity to competitive intelligence or the replication of the agent without the vendor. The **External Attacker** ($\mathcal{A}_2$) reaches only the public API of the agent and relies on prompt injection, jailbreaking, and social engineering through the model’s input–output interface. The **Malicious Insider** ($\mathcal{A}_3$) is a member of either organization with partial access, for instance to development environments, CI/CD pipelines, or logging systems, who attempts to exfiltrate prompt intellectual property.

3.3 Security Requirements and Attack Mapping

The architecture must provide three guarantees. *Prompt confidentiality* requires that the plaintext prompt never be accessible inside the customer account, neither at rest nor in transit. *Output integrity* requires that responses reflect the vendor’s prompt and that any tampering be detectable. *Forensic traceability* requires that every response carry an invisible per-request fingerprint, so that leaked content can be attributed to an account, a session, and a request; the fingerprint is an attribution mechanism built into the response path, and its robustness against Unicode normalization, deliberate removal, or collusion is not claimed here. Table 1 maps each attack vector to the adversaries able to execute it, to the guarantee it targets, and to the defense layer that counters it. The table uses the abbreviations Hash-based Message Authentication Code (HMAC), man-in-the-middle (MITM), and zero-width character (ZWC).

Trust assumptions. The analysis rests on four assumptions: (1) the vendor’s Identity and Access Management (IAM) configuration is correct; (2) all cross-account communication uses Transport Layer Security (TLS) 1.2 or later; (3) the container runtime isolates process memory; and (4) the LLM API does not retain prompts beyond the session. Assumption (1) is weaker than the hardware attestation of a TEE, but it holds on every cloud provider without additional

Table 1: Attack vectors, adversary capabilities, and defense layer mapping.

Attack vector	Adv.	Goal	Defense
Console/API inspection	$\mathcal{A}_1$	Conf.	L1: Cross-acct isolation
Container reverse eng.	$\mathcal{A}_1$	Conf.	L1: Build-time secret
Direct gatekeeper call	$\mathcal{A}_1$	Conf.	L1: HMAC + account binding
Request replay	$\mathcal{A}_1, \mathcal{A}_2$	Conf.	L1: Timestamp + nonce
MITM tampering	$\mathcal{A}_1$	Integ.	L1: Body hash in HMAC
Prompt injection / jailbreak	$\mathcal{A}_2$	Conf.	L2: Risk scoring
Multi-turn extraction	$\mathcal{A}_2$	Conf.	L2+L3: Score + filter
Output exfiltration	$\mathcal{A}_2, \mathcal{A}_3$	Conf.	L3: Canary + detection
Unauthorized redistribution	$\mathcal{A}_3$	Trace.	L3: ZWC fingerprint
Encoding evasion	$\mathcal{A}_2$	Conf.	L2: Encoding detectors
Typoglycemia attacks	$\mathcal{A}_2$	Conf.	L2: Normalization
Monitoring evasion	$\mathcal{A}_1$	All	L4: Vendor-side logs

hardware. Assumption (3) delimits the scope of the model: an adversary who attaches a debugger to the agent process, inspects its runtime memory, or otherwise compromises the process can read the decrypted prompt while it is in use, and no software-only layer of the architecture prevents this. Such an adversary is outside the threat model; the per-request fingerprint of Layer 3 makes the resulting disclosure attributable rather than preventable, and deployments that must resist it require hardware-backed isolation.

4 Defense-in-Depth Architecture

The architecture composes four defense layers into a sequential trust chain, as network security pipelines chain firewalls, Intrusion Detection Systems (IDS), Web Application Firewalls (WAF), and Security Information and Event Management (SIEM) systems. Figure 1 shows the two-account deployment model. The *Vendor Account* contains the trusted components: a secrets store whose prompts are encrypted with a Key Management Service (KMS) key, a serverless gatekeeper in charge of authentication and prompt delivery, a container registry, and a central event bus. The *Customer Account* contains the deployed components: the containerized agent, an API gateway with rate limiting, and an event-forwarding mechanism. The agent obtains its prompt at runtime and keeps it exclusively in process memory, which enforces the invariant on which the design depends: the system prompt is never persisted in customer-controlled storage and exists only transiently during inference.

Preserving prompt confidentiality across accounts also requires a sequence of coordinated defenses applied at different stages of the interaction. Figure 2 shows this pipeline: Layer 1 secures prompt delivery through cryptographic request validation, Layer 2 applies OWASP-aligned input validation before inference, Layer 3 filters model outputs and embeds forensic fingerprints, and Layer 4 centralizes cross-account monitoring. Figure 3 details the internal mechanism of each layer; the following subsections describe them in turn, followed by the prompt-level hardening that complements them.

4.1 Layer 1: Cryptographic Prompt Delivery

This layer guarantees that only authenticated agents can retrieve prompts and that requests can be neither tampered with nor replayed, through a cryptographic binding between the request payload, the originating account, and temporal information. Each request carries an HMAC with SHA-256 (HMAC-SHA256) signature over a structured message:

$$m = ts \parallel nonce \parallel account_id \parallel SHA256(body) \quad (1)$$

The signature is $\sigma = \text{HMAC-SHA256}(k, m)$, where k is a pre-shared secret key; the construction protects the integrity of the body, binds the request to one account, and provides freshness through the timestamp and the nonce. Before releasing the prompt, the gatekeeper checks four properties: (1) the timestamp lies within a bounded window, set to five minutes in the reference implementation; (2) the nonce has not been observed before, which a short-lived cache enforces within that window; (3) the account identifier belongs to an allow-list; and (4) the request body matches the signed hash. All comparisons use constant-time operations. The gatekeeper additionally enforces a per-account request rate and raises an alert after repeated failed attempts from the same account, and it returns with the prompt an integrity hash (SHA-256 of the prompt content) that the agent verifies on receipt. The signing key is supplied at container build time through a build secret that is never written to an image layer, so it cannot be recovered from the image history. At the application layer, the mechanism therefore provides request authenticity, payload integrity, and replay protection, even against adversaries with visibility into the execution environment.

4.2 Layer 2: OWASP-Aligned Input Validation

This layer mitigates prompt injection and adversarial manipulation by screening user inputs before they reach the model, through a quantitative risk assessment aligned with the OWASP LLM Top 10 [13] and with the complementary taxonomy of the National Institute of Standards and Technology (NIST) [35, 12]. Every incoming message receives a risk score computed from the patterns it matches. The patterns correspond to known attack strategies, such as instruction override, jailbreak personas, prompt extraction requests, and structural injection markers, and are grouped into four severity tiers with additive weights: critical (+30), high (+20), medium (+10), and low (+5); structural heuristics add smaller increments for unusually long inputs, an excessive number of line breaks, or embedded markup tags. Inputs whose score reaches a fixed threshold, set to 20, are blocked with a fixed refusal message; inputs in the medium band are logged as suspicious and allowed through. The threshold follows from the weights: a score of 20 corresponds

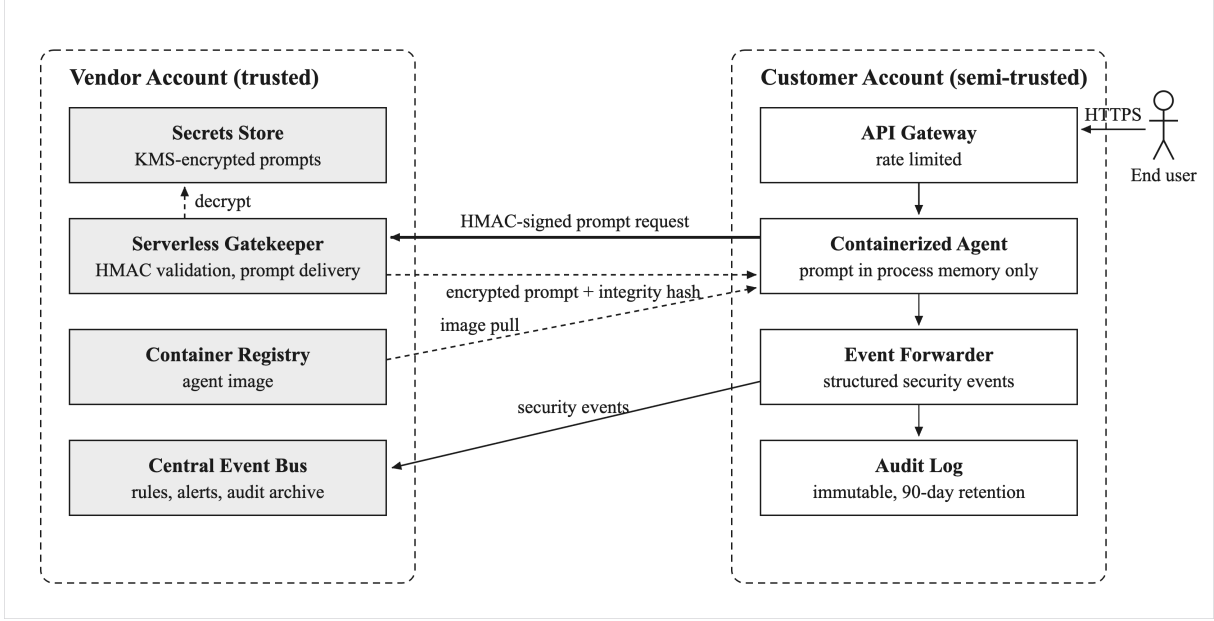


Figure 1: Two-account architecture for cross-domain prompt protection.

to one high-severity pattern or to two independent medium-severity signals, so a single reference to an encoding scheme is tolerated while combinations of weaker signals trigger blocking; the empirical support for this operating point is reported in Section 7.2. Two specialized detectors extend coverage beyond explicit patterns. A typoglycemia detector matches deliberately misspelled variants of security-critical keywords, such as “ignroe” or “systme”, against a dictionary of known variants, and two or more variants add +20 to the score; an encoding detector flags payloads hidden through base64, hexadecimal, or Unicode escape sequences, as well as explicit requests to decode ROT13 or similar transformations, and any detected encoding adds +20. Before the message is forwarded to the model, a sanitization step replaces known injection prefixes, such as override commands or fake role markers, with a neutral placeholder.

4.3 Layer 3: Output Filtering and Watermarking

This layer prevents the disclosure of sensitive information in model responses and enables forensic attribution of generated content, through three complementary mechanisms. First, responses are inspected for prompt leakage with a set of compiled regular expressions that recognize meta-instructions, role definitions, jailbreak success indicators, and configuration dumps, and with a sliding-window comparison against the delivered prompt: the prompt is divided into 50-character chunks, and a response that reproduces any chunk is treated as a leak. Responses that trigger either check are replaced by a fixed safe message. Second, canary tokens are embedded in the prompt itself: each canary is an HTML comment that

contains a truncated SHA-256 hash derived from the account identifier and a timestamp, placed before and after the prompt body, invisible in normal operation, and producing an alert and a blocked response whenever it appears in an output. Third, every response is augmented with a steganographic fingerprint built from zero-width Unicode characters (ZWC) [33, 31]. The fingerprint is the first eight hexadecimal digits of the SHA-256 hash of the account, session, and request identifiers, mapped to sequences of the characters U+200B, U+200C, and U+200D, split into three fragments, and inserted at three random word boundaries of the response (or prepended when the response has fewer than five words), so that it remains imperceptible and resists naive removal. The same module provides Personally Identifiable Information (PII) detection and redaction routines, which the static test suite exercises. The layer thus filters prompt leakage at generation time and embeds an attribution mechanism for content that is exfiltrated; the fingerprint was verified to be embedded and decodable, but its resistance to normalization, deliberate stripping, or collusion was not evaluated.

4.4 Layer 4: Cross-Account Monitoring

This layer provides system-wide observability and coordinated detection of security events across the trust boundary. Monitoring is centralized in the vendor-controlled environment, so adversaries operating inside the customer account cannot suppress or alter security-relevant signals. Events generated by all layers are forwarded to a central event bus, where they are aggregated and correlated: authentication failures, replay attempts, body-hash mismatches, and rate-limit

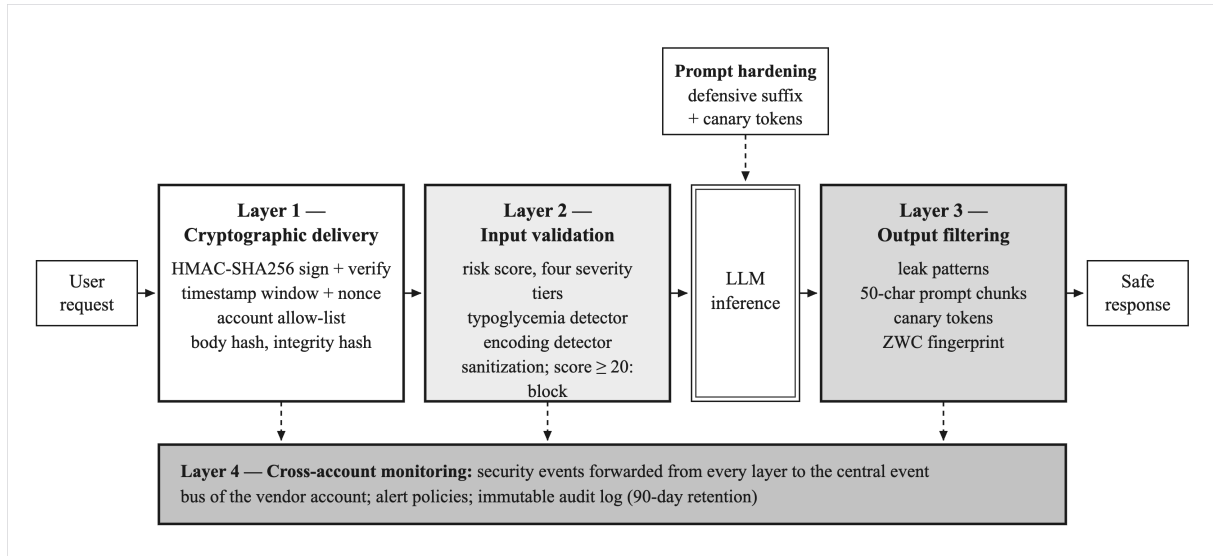


Figure 2: Defense-in-depth pipeline: Layers 1 and 2 act before model inference, Layer 3 acts on the model output, and Layer 4 receives the security events of every layer.

violations from Layer 1; input-validation triggers from Layer 2; and leakage or canary detections from Layer 3. The customer account additionally forwards control-plane events that indicate tampering by a privileged principal, such as modifications of the agent function, changes to the IAM roles that protect it, direct access attempts to the secrets store, alterations of the audit trail, or activation of model-invocation logging. Alerting policies escalate suspicious activity: in the reference implementation, three gatekeeper authentication failures within five minutes, five rate-limit rejections within five minutes, or any replay attempt raise an alert, and every input-validation block in the customer account is forwarded within one minute. All events are also written to an immutable audit log with a 90-day retention period, so that incidents can be reconstructed even after a partial compromise.

4.5 Prompt-Level Hardening

Besides the four layers, the prompt delivered by the gatekeeper is hardened in memory before the agent is instantiated. In addition to the canary tokens of Layer 3, a defensive suffix appends eight explicit rules that instruct the model never to reveal, paraphrase, or encode its instructions, to decline extraction attempts framed as roleplay, hypotheticals, or translation, and to answer such attempts with a fixed refusal sentence. This control is inexpensive and requires no infrastructure, but its effect depends entirely on the instruction-following behavior of the underlying model, which the architecture does not control. For that reason it is not counted as a defense layer: it is treated as an explicit factor in the multi-model evaluation and in the ablation study, and the reproducibility environment provides alternative variants of the suffix designed for small open-weight models (Section 5.2).

5 Reference Implementation and Reproducibility

The implementation is a central part of the contribution: it shows that the architecture can be built from standard managed services, and it provides the artifacts used in every experiment of this article. The reference implementation and the experimental artifacts are publicly available¹.

5.1 Cloud Deployment

The system is built on serverless services and is defined entirely as Infrastructure-as-Code (IaC) with Terraform. Two environments correspond to the two accounts of the model. The central environment provisions the vendor-side components: the encrypted secrets store and its KMS key, the gatekeeper function behind an HTTP endpoint, the container registry, the central event bus with its rules, the alerting topics, and the audit storage. The client environment provisions the customer-side components: the agent function packaged as a container image, the API gateway with throttling, the event forwarder, and the detection rules of Section 4.4. The architecture relies only on service primitives that every major provider offers (managed secrets and keys, serverless functions, container registries, cross-account event routing, and immutable log storage), with direct equivalents on Azure and Google Cloud for each AWS service of the reference implementation.

The agent is a Python application built on an agent framework with tool calling; it exposes an HTTP interface for health checks, tool listing, and invocation, runs

¹<https://github.com/dpetrocelli/defense-in-depth-agent-ip>

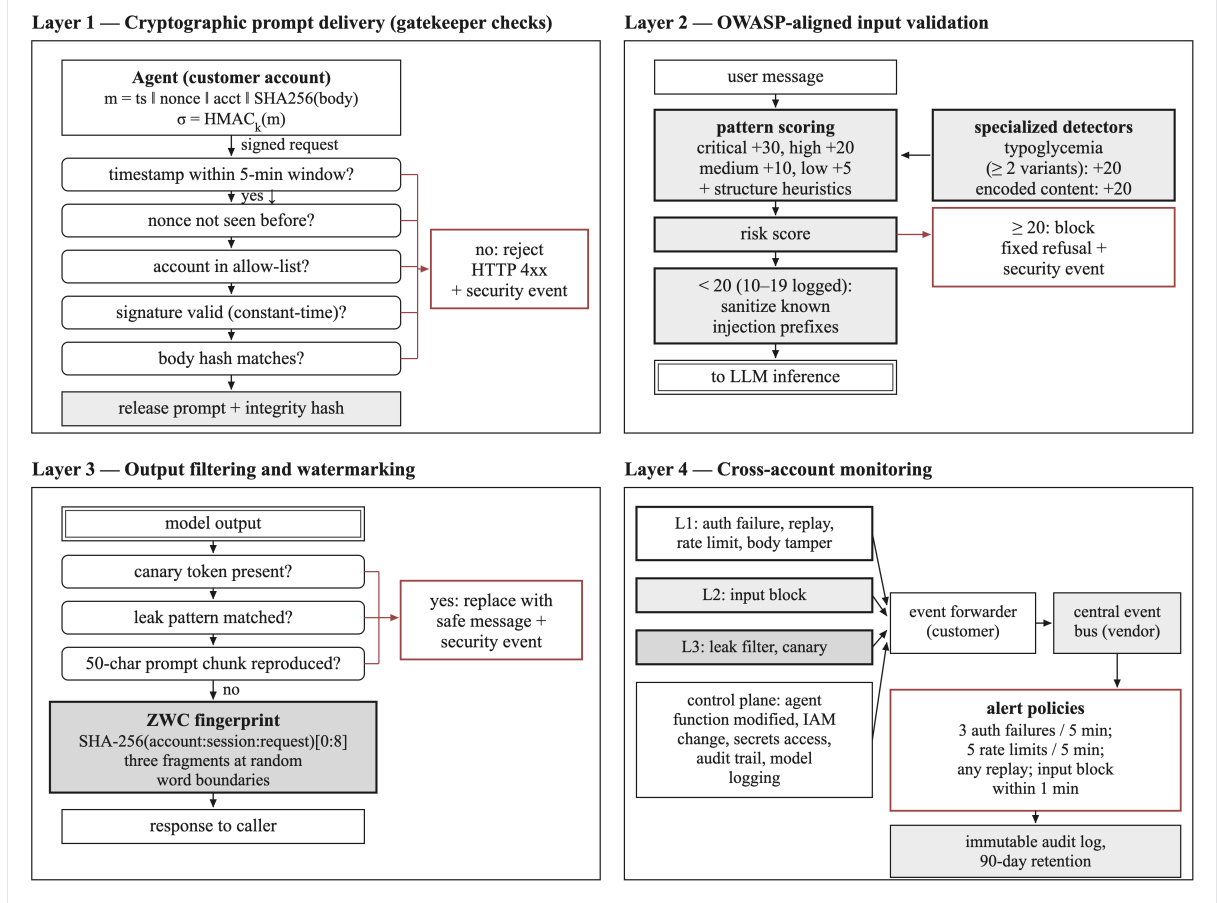


Figure 3: Internal mechanism of each defense layer as implemented in the reference implementation: validation order of the gatekeeper (Layer 1), scoring and blocking decision (Layer 2), output checks and fingerprint embedding (Layer 3), and event sources, forwarding, alerting, and audit (Layer 4).

both as a serverless function and as a local process, and ships with four demonstration tools (arithmetic, current time, string utilities, and Fibonacci numbers). In the cloud deployment the model is Amazon Nova Lite, invoked through the provider’s managed inference service. Layer 1 is split between the gatekeeper function in the vendor account and a signing client inside the agent; Layers 2 and 3 are a self-contained filtering module inside the agent; Layer 4 combines the event forwarder, the event-bus rules, and the alarms of both accounts. Deployment proceeds in three phases: *build*, in which the container image is produced with the signing key mounted as a build secret that is absent from every image layer; *provisioning*, in which the customer-side resources are instantiated through cross-account policies that let the customer function pull the image without granting that permission to the customer administrator; and *activation*, in which the agent retrieves its prompt from the gatekeeper, verifies the integrity hash, applies the hardening of Section 4.5, and keeps the result only in process memory. Before serving traffic, a fail-closed pre-flight check verifies that model-invocation logging is disabled in the customer account, which enforces trust assumption (4) of

Section 3.

Every defense mechanism is governed by a feature flag exposed as an environment variable of the agent: `USE_GATEKEEPER` (Layer 1), `ENABLE_INPUT_VALIDATION` and `BLOCK_HIGH_RISK_INPUTS` (scoring and blocking in Layer 2), `ENABLE_RESPONSE_FILTER` and `ENABLE_WATERMARKING` (Layer 3), and `ENABLE_PREFLIGHT_CHECK`. All flags default to enabled, and disabling one changes only the corresponding mechanism; for the experiments of this article, the defensive suffix, the input sanitization step, and canary detection were placed under flags of their own. This configuration surface is the basis of the ablation study of Section 7.4.

5.2 Local Reproducibility Environment

To allow experimentation without cloud accounts, the repository provides a Docker Compose environment that combines LocalStack for cloud-service emulation and Ollama for local LLM inference. A single-account variant runs the agent against an emulated secrets store with the gatekeeper disabled. A multi-account variant reproduces the two-account model with two isolated

LocalStack instances: a central instance hosts the gatekeeper function behind an API endpoint, the secrets store, the KMS key, and an alerting topic with a queue that allows Layer 4 alerts to be verified, and a client instance provides a separate identity for the agent. The gatekeeper executes the same code as in the cloud deployment, so all four layers are exercised end to end without code modifications. Any model served by Ollama can be selected through a configuration variable; the agent uses the native Ollama driver of the agent framework, which transmits the system prompt and the tool definitions as separate fields of the request, since a generic proxy library used earlier silently dropped the system message when tools were present. The environment also provides three alternative defensive suffixes for small open-weight models and the experimental tooling (test suite, corpus and runner, false-positive corpus, and latency benchmark).

6 Experimental Methodology

6.1 Metrics, Corpus, and Environments

Security effectiveness is measured on two levels. Controlled penetration testing verifies each mechanism against a fixed suite of 63 test cases in 12 categories aligned with the OWASP LLM Top 10 [13] and the NIST taxonomy of adversarial machine learning [35]; every test specifies attack vector, targeted goal, expected layer, payload, and pass/fail criterion. Live evaluation submits a corpus of 42 attack queries in eight categories of increasing sophistication, naive extraction (8), instruction override (5), jailbreak and persona change (7), encoded payloads (4), typoglycemia (4), social engineering (5), indirect injection through fake system markers (4), and multi-turn escalation (5), together with 23 legitimate queries in three length groups (10 short, 10 medium, 3 long), to a deployed agent. It measures the *block rate*, the share of attack queries that do not obtain a substantive answer, the *false-positive rate* (FPR), the share of legitimate queries that are blocked, and, in the new experiments, the *attack success rate* (ASR), the share of attack queries whose final response leaks protected content, so that blocking and leakage are measured separately. In the live evaluation of Section 7.2, an attack counts as blocked when the request is rejected, the response is empty, or the response is a refusal; the new experiments refine this criterion with the per-request classification of Section 6.2. Performance overhead is measured as per-layer latency at the 50th, 95th, and 99th percentiles (P50, P95, P99) and as throughput under load; deployability as monthly infrastructure cost on three providers. Two environments are used: the two-account AWS deployment of Section 5, with Amazon Nova Lite as the underlying model, for the live evaluation, the performance measurements, and the cost analysis; and the multi-account Docker Compose stack of Section 5.2, which runs the same agent and gatekeeper code against

open-weight models served by Ollama, for the multi-model evaluation and the ablation study. Both use the test prompts of the corpus rather than a production prompt.

6.2 Multi-Model Evaluation Protocol

The multi-model evaluation answers RQ1 by holding every element of the pipeline fixed and varying only the underlying LLM, so that differences in outcome can be attributed to the model. Nine models from six families are evaluated, covering both deployment modes of the architecture. Amazon Nova Lite (`amazon.nova-lite-v1:0`) is invoked through Amazon Bedrock and serves as the reference model of the live evaluation. The eight open-weight models are served by Ollama 0.33.3 in the local environment, with the quantization published under each identifier: Llama 3.2 1B (`llama3.2:1b`, Q8_0), Llama 3 8B (`llama3:8b`, Q4_0), Gemma 3 4B (`gemma3:4b`), Gemma 4 E4B (`gemma4:e4b`, 4B effective parameters), Phi-4 Mini (`phi4-mini`, 3.8B), Mistral 7B (`mistral:7b`), Qwen 3.5 4B (`qwen3.5:4b`), and Qwen 3.5 9B (`qwen3.5:9b`, 9.6B), the last six at Q4_K_M; seven of them ran on a single laptop GPU (RTX 4060, 8 GB) and Gemma 4 E4B on an RTX 3090. All models run the same agent and gatekeeper code, and serving conditions are constant within each model, which is what the comparison across configurations requires.

The following factors are fixed across models. The base system prompt is the medium-length prompt of the corpus (about 600 characters), whose closing sentence is a minimal confidentiality instruction that forms part of the base prompt in every configuration; it is hardened with the production defensive suffix and the canary tokens of Section 4.5. The four demonstration tools were registered for all models except Gemma 3 4B and Llama 3 8B, whose runs were executed without tool registration. Decoding is deterministic (temperature 0, nucleus sampling disabled, at most 1,024 output tokens on Ollama). All four layers are enabled, which corresponds to configuration C4 of Table 2. Every query is submitted in a fresh conversation, except the multi-turn escalation category, whose five queries form one session and are submitted in their listed order. Each model is evaluated in three independent runs, reported as mean and standard deviation. Requests that end in a transport error or a timeout are counted as errors and excluded from the denominators; the complete campaign comprises 10,530 requests, 7 of which ended in errors.

The outcome of every request is classified by the mechanism that determined it: *L2 block* (risk score at or above the threshold), *model refusal* (the model declines before any filtering), *L3 filter* (the response filter replaced the output), *canary* (a canary token appeared in the output), *pass* (a substantive response reached the caller), or *error*. Because Layer 2 blocks, canary

Table 2: Ablation configurations. DP denotes the defensive suffix of Section 4.5; Sc and BI the scoring and the blocking of Layer 2; Sa the input sanitization; Fi, Ca and WM the response filter, the canary detection and the watermark of Layer 3. Layer 1 and Layer 4 remain active in every configuration and are verified separately.

Config.	DP	Sc	BI	Sa	Fi	Ca	WM
C0	—	—	—	—	—	—	—
C1	✓	—	—	—	—	—	—
C2	✓	✓	✓	✓	—	—	—
C3	✓	—	—	—	✓	✓	✓
C4	✓	✓	✓	✓	✓	✓	✓
C4 [†]	✓	✓	—	✓	✓	✓	✓

detections, and model refusals return the same refusal sentence, the label is recorded by the agent as a structured audit field rather than inferred from the response text; the raw model output produced before Layer 3 is recorded as well. Leakage is assessed offline with the same criterion that Layer 3 applies at runtime: a response leaks if it contains a canary token or reproduces any 50-character chunk of the base prompt, ignoring case. The oracle is applied both to the final response, which yields the ASR, and to the raw model output, which yields the *pre-filter leakage* of the model. The block rate is the share of attack queries whose outcome is an L2 block, a model refusal, an L3 filter, or a canary detection; the FPR is the share of legitimate queries with any of these outcomes. Results are reported per model, with the attribution of outcomes to mechanisms (Table 4), and per attack category (Table 5).

6.3 Defense-Layer Ablation Protocol

The ablation study answers RQ2 by measuring how the security outcome changes when individual defenses are removed. The layers are not interchangeable classifiers: Layer 1 governs authentication, request integrity, account binding, and replay protection, and Layer 4 governs monitoring, auditability, and coordinated response; neither acts on the content of a request or a response, so removing them does not change the outcome of the attack corpus. Both are kept active in every configuration and verified separately, with attack cases of their own, in Table 8. The ablation therefore varies the *content-path protections*, the mechanisms that act on the input–output path of a request: the defensive suffix, Layer 2, and Layer 3. Layer 1 and Layer 4 are validated functionally rather than as factors of the ablation.

Table 2 summarizes the configurations. C0 disables every content defense and measures the floor set by the model itself, since the base prompt retains only its closing confidentiality sentence; C1 adds the defensive suffix alone; C2 and C3 add Layer 2 and Layer 3, respectively, on top of the suffix; C4 is the deployed pipeline and coincides with the condition of the multi-model evaluation; and C4[†] keeps the pipeline of C4

but runs Layer 2 in detect-only mode, in which the risk score is computed and recorded while blocking is disabled, so that every attack reaches the model and Layer 3 and the verdicts of both layers are observed on the same request. Input sanitization and canary detection follow the Layer 2 and Layer 3 settings, respectively. All configurations are evaluated for the nine models with the fixed factors, repetitions, classification, and metrics of Section 6.2. The contribution of a mechanism is the change in block rate and ASR between configurations that differ only in that mechanism: the defensive suffix from C0 to C1, Layer 2 from C1 to C2 and from C3 to C4, and Layer 3 from C1 to C3 and from C2 to C4. Complementarity is measured on C4[†] by labeling each attack query with the verdict of Layer 2 (would block or pass) and with its downstream outcome (intercepted by Layer 3 or by a canary, refused by the model, or passed), which yields Table 7; queries that pass Layer 2 and are intercepted by Layer 3 are the direct evidence of complementarity, and queries that pass both are evaluated with the leakage oracle.

Layer 1 is verified with deterministic cases executed against the gatekeeper of the multi-account environment, which runs the production code: a valid signed request must return the prompt with a valid integrity hash, and each violated property must produce the response that follows from the validation order of the gatekeeper without disclosing prompt content. Layer 4 is verified by triggering each event class: the gatekeeper-side classes (repeated authentication failures, replay, rate limiting) in the same environment, whose alert topic is read through a subscribed queue, and the agent-side classes (Layer 2 blocks, Layer 3 filters, canary detections) and the customer-side control-plane rule on the two-account cloud deployment, where delivery is read from the archive log of the central event bus and from the invocation metrics of the forwarding rules, because the alarms of Layer 4 are cloud metric alarms that the local environment does not emulate.

7 Security Evaluation

7.1 Controlled Penetration Testing

All 63 test cases of the controlled suite passed: 44 input-side tests (direct and multi-turn extraction, jail-break and “Do Anything Now” (DAN) personas, instruction override, prompt injection, encoding and typosglucemia evasion, social engineering), 15 output-side tests (output leakage, Personally Identifiable Information (PII) exfiltration, response safety), and 4 replay and authentication bypass tests, which were rejected with Hypertext Transfer Protocol (HTTP) 403 responses that disclosed no information. The typosglucemia detector blocked obfuscated variants that defeat a naive regular-expression approach, the sliding-window comparison of Layer 3 detected partial leak-

Table 3: Live deployment: attack detection by category (42 attacks, 23 legitimate queries). FP = false positive.

Attack category	Blocked	Rate	Defense
Override commands	5/5	100%	L2 (risk score)
Jailbreak / DAN	7/7	100%	L2 (risk score)
Encoded payloads	4/4	100%	L2 (encoding det.)
Typoglycemia	4/4	100%	L2 (typo det.)
Social engineering	5/5	100%	L2+L3
Indirect injection	4/4	100%	L2 (structural)
Naive extraction	6/8	75%	L2+L3+defensive prompt
Multi-turn	3/5	60%	L2+L3 (escalation)
Total attacks	38/42	90.5%	
Legitimate (FP)	0/23	0.0%	

age of fragments of 50 or more characters that the pattern-based check alone did not capture, and eight boundary-condition tests confirmed that attacks crafted to lower their Layer 2 score were still mitigated by Layer 3.

7.2 Live Two-Account Deployment

To validate the architecture under realistic conditions, the full four-layer stack was deployed across two separate cloud accounts, vendor and customer, and exposed to the 42 attack queries and the 23 legitimate queries of Section 6.1. Table 3 reports the outcome per category: every category was fully blocked except naive extraction (6 of 8) and multi-turn escalation (3 of 5), for a total of 38 of 42 attacks (90.5%) and no false positive on the 23 legitimate queries. The four queries that were not blocked are reconnaissance probes that obtain no prompt content.

Multi-turn escalation has the lowest rate because input validation is stateless: escalation sessions spread their intent over turns that are individually benign, only one of the five sessions contains a turn that reaches the blocking threshold by itself, and the sessions that were blocked were intercepted by the escalation heuristics of Layer 2 and the output-side filter of Layer 3 once the responses approached protected content; cumulative session-level scoring would close this gap. A sensitivity analysis over the corpus supports the operating point of Layer 2: the coverage of the score-based detector is stable between thresholds 15 and 20 (20 of 42 attacks) and falls sharply at 25 (12 of 42), while no legitimate query scores above 5. The detector set was calibrated iteratively against a preliminary live run, which raised the overall blocking rate from 59.5% to the final 90.5% while keeping false positives at zero. Under concurrent load, throughput remained stable up to 50 concurrent requests (13.8 requests per second) and degraded at higher concurrency because of infrastructure limits.

7.3 Multi-Model Evaluation

Table 4 reports the full pipeline across the nine models under the protocol of Section 6.2; since the corpus, the prompt, the hardening, the decoding parameters,

and the four layers are identical for every model, the variation across rows is attributable to the underlying LLM.

No final response met the verbatim leakage criterion: the ASR is 0.0% for all nine models, that is, none of the 1,134 attack requests of this condition returned a canary token or a 50-character chunk of the base prompt to the caller. The block rate, in contrast, depends strongly on the model: it ranges from 62.7% for Llama 3.2 1B to 93.7% for Nova Lite, with Mistral 7B, Gemma 4 E4B, Llama 3 8B, and Qwen 3.5 9B above 84% and Phi-4 Mini and Qwen 3.5 4B below 75%. The value of Nova Lite is consistent with the 90.5% of the live deployment. Standard deviations across runs do not exceed 4.1 points, so the differences between models are not run-to-run variation.

The attribution columns separate the model-independent part of this result from the model-dependent part. Layer 2 blocked exactly 75 of the 126 attack requests of every model, that is, 25 of the 42 attacks in each run (59.5%): the risk scorer and the specialized detectors are deterministic, so their contribution does not depend on the model, and it coincides with the 59.5% floor observed during the calibration of Section 7.2. Everything above that floor comes from the refusals of the model, which follow the defensive suffix, and from Layer 3. Refusals account for 43 of the remaining 51 requests for Nova Lite and for none of them for Llama 3.2 1B, with the other models in between (13 to 33); Layer 3 intercepted between 0 and 7 responses per model. The requests that were neither blocked nor refused, from 8 for Nova Lite to 47 for Llama 3.2 1B, were answered without reproducing protected content. Two caveats apply: a refusal is recognized by the fixed refusal sentence, so a model that declines in its own words counts as having passed, and passing without leak is judged by the verbatim oracle.

Table 5 shows where the model matters. Four of the eight naive extraction attacks, four of the five override commands, four of the seven jailbreaks, three of the four encoded payloads, two of the four typoglycemia variants, four of the five social-engineering attacks, three of the four indirect injections, and one of the five multi-turn turns reach the blocking threshold of Layer 2 on their own; this floor is visible in the column of Llama 3.2 1B, which coincides with it in every category except multi-turn escalation, where Layer 3 adds four interceptions. The remaining attacks are intercepted only when the model refuses them or Layer 3 filters the response: naive extraction and typoglycemia are fully covered by four models each, whereas multi-turn escalation remains the weakest category for every model except Llama 3 8B (from 3 of 15 turns for Gemma 3 4B and Phi-4 Mini to 15 of 15), although every escalation session was intercepted in at least one turn for all nine models.

The false-positive rate is also model-dependent, and it originates in the defensive suffix rather than

Table 4: Multi-model evaluation with the full pipeline (configuration C4). Left: block rate, attack success rate (ASR, leaked final responses), and false-positive rate on the 23 legitimate queries, as mean $\pm$ standard deviation over three runs, excluding requests that ended in errors. Right: attribution of the 126 attack requests of each model (42 per run) to the mechanism that determined their outcome. No canary token was reproduced and no leaked response reached the caller for any model.

Model	Block rate (%)	ASR (%)	FPR (%)	L2 block	Model refusal	L3 filter	Passed (no leak)
Nova Lite	93.7 $\pm$ 1.4	0.0 $\pm$ 0.0	7.2 $\pm$ 2.5	75	43	0	8
Llama 3.2 1B	62.7 $\pm$ 1.4	0.0 $\pm$ 0.0	0.0 $\pm$ 0.0	75	0	4	47
Llama 3 8B	85.7 $\pm$ 0.0	0.0 $\pm$ 0.0	4.3 $\pm$ 0.0	75	30	3	18
Gemma 3 4B	81.0 $\pm$ 0.0	0.0 $\pm$ 0.0	15.9 $\pm$ 2.5	75	27	0	24
Gemma 4 E4B	88.1 $\pm$ 4.1	0.0 $\pm$ 0.0	11.6 $\pm$ 2.5	75	33	3	15
Phi-4 Mini	73.8 $\pm$ 0.0	0.0 $\pm$ 0.0	13.0 $\pm$ 0.0	75	18	0	33
Mistral 7B	89.7 $\pm$ 1.4	0.0 $\pm$ 0.0	13.0 $\pm$ 0.0	75	31	7	13
Qwen 3.5 4B	69.8 $\pm$ 2.7	0.0 $\pm$ 0.0	0.0 $\pm$ 0.0	75	13	0	38
Qwen 3.5 9B	84.9 $\pm$ 1.4	0.0 $\pm$ 0.0	0.0 $\pm$ 0.0	75	32	0	19

in Layer 2, which blocked no legitimate query in any configuration of the campaign. Five models answered between one and four of the 23 legitimate queries with the fixed refusal sentence, mostly short generic questions such as “What time is it?”, which yields FPRs between 4.3% (Llama 3 8B) and 15.9% (Gemma 3 4B); Llama 3.2 1B and the two Qwen models produced no false positive. The 11.6% of Gemma 4 E4B is of a different nature: the model returned empty responses to the three long legitimate queries, which the classification counts as refusals, also without any defense (C0), so this rate reflects the model runtime rather than the defenses. Finally, the pre-filter leakage shows what Layer 3 removed: with the full pipeline, 29.4% of the raw outputs that reached the model reproduced a chunk of the prompt for Mistral 7B, 5.9% for Llama 3.2 1B, Llama 3 8B, and Gemma 4 E4B, and 3.9% for Qwen 3.5 4B, and none of these fragments reached the caller.

In answer to RQ1, the architecture kept the attack success rate, as measured by the verbatim criterion, at zero for every model family, while the block rate, which measures how early an attack is stopped, varied by 31 points between the weakest and the strongest model; the variation is entirely attributable to the defensive suffix and the refusal behavior of the model, since the contribution of Layer 2 was identical for all models.

7.4 Defense-Layer Ablation Study

Table 6 gives block rate, ASR, and FPR for every configuration of Table 2 and every model. Without content defenses (C0) no attack is blocked, and the intrinsic leakage of the models ranges from 0.0% (Qwen 3.5 9B) to 34.1% (Llama 3 8B), 13.0% on average. The defensive suffix alone (C1) raises the mean block rate to 55.0% and lowers the mean ASR to 5.2%, but its effect is the most model-dependent of the study: the block rate goes from 6.3% for Llama 3.2 1B to 93.6% for Nova Lite, and Mistral 7B and Qwen 3.5 4B still leak in 19.0% and 15.9% of the attacks. Adding Layer 2

(C2) raises the mean block rate by 24.0 points. Layer 2 blocks the same 25 attacks per run for every model, so its gain is largest where the model refuses least: +54.0 points for Llama 3.2 1B, +42.1 for Qwen 3.5 4B, and between +31 and +36 for Gemma 3 4B, Phi-4 Mini, and Llama 3 8B, against +0.1 for Nova Lite and -0.8 for Gemma 4 E4B, whose refusals already cover most of what Layer 2 blocks. Layer 2 reduces the mean ASR to 2.1% but does not eliminate it: Mistral 7B leaks in 12.7% of the attacks and four other models in 0.8–2.4%, because Layer 2 stops only the attacks that match its patterns and the model answers the rest. Adding Layer 3 instead (C3) raises the mean block rate by only 2.7 points, but it brings the ASR to 0.0% for all nine models, as does the full pipeline (C4).

The two layers therefore contribute different properties: Layer 2 determines how many attacks are stopped before inference, at no cost in false positives; Layer 3 determines whether what the model generates reaches the caller, and it closes the residual leakage left by the model and by Layer 2. The marginal contributions summarize this asymmetry: +24.0 and +23.3 points of block rate for Layer 2 against +2.7 and +2.0 for Layer 3, and -5.2 and -2.1 points of ASR for Layer 3 against -3.1 and 0.0 for Layer 2. The false positives of the pipeline come from the defensive suffix (mean FPR 6.0% in C1) and, marginally, from Layer 3, which filtered one legitimate response of Qwen 3.5 9B; Layer 2 added none.

Table 7 shows the two layers on the same requests. Of the 17 attack queries per run that pass Layer 2, Layer 3 intercepted the response for Qwen 3.5 4B (10 of 51 requests over the three runs), Mistral 7B (9), Llama 3.2 1B (3), and Nova Lite (2); for the other five models, the queries that passed Layer 2 were either refused by the model or answered without leaking. Layer 3 also intercepted responses to attacks that Layer 2 would have blocked (11 for Qwen 3.5 4B, 6 for Mistral 7B, 3 for Gemma 4 E4B), which is redundant coverage. No request passed both layers with a leaked response for any model: in detect-only mode the block rate falls to between 4.8% (Llama 3.2 1B)

Table 5: Block rate per attack category and model with the full pipeline (configuration C4), as blocked/total over the three runs.

Attack category	Nova Lite	Llama 3.2 1B	Llama 3 8B	Gemma 3 4B	Gemma 4 E4B	Phi-4 Mini	Mistral 7B	Qwen 3.5 4B	Qwen 3.5 9B
Naive extraction (8)	24/24	12/24	24/24	21/24	24/24	24/24	21/24	15/24	17/24
Override commands (5)	15/15	12/15	12/15	12/15	12/15	12/15	15/15	12/15	15/15
Jailbreak / DAN (7)	21/21	12/21	18/21	18/21	18/21	15/21	21/21	13/21	15/21
Encoded payloads (4)	12/12	9/12	9/12	12/12	12/12	9/12	12/12	9/12	9/12
Typoglycemia (4)	12/12	6/12	6/12	9/12	12/12	6/12	12/12	6/12	12/12
Social engineering (5)	15/15	12/15	15/15	15/15	15/15	15/15	15/15	12/15	15/15
Indirect injection (4)	12/12	9/12	9/12	12/12	12/12	9/12	9/12	12/12	12/12
Multi-turn escalation (5)	7/15	7/15	15/15	3/15	6/15	3/15	8/15	9/15	12/15
Total (42)	118/126	79/126	108/126	102/126	111/126	93/126	113/126	88/126	107/126

and 95.9% (Nova Lite), yet the ASR remains 0.0%. The cells “Layer 2 would block, passes” quantify what the deterministic layer adds beyond the model: 72 of 75 such requests for Llama 3.2 1B, 48 for Qwen 3.5 4B, and 42 for Llama 3 8B, against none for Nova Lite, so for the weakest models Layer 2 is the only mechanism that keeps these attacks from reaching inference. For the cloud reference model the defensive suffix alone already intercepts 117 of the 126 attack requests, and multi-turn escalation is the only category that remains partially covered in every configuration (7 to 9 of 15 turns), consistent with the stateless scoring discussed in Section 7.2; the per-category ablation of every model is provided in the repository.

Table 8 reports the verification of the layers outside the ablation. All eight Layer 1 cases produced the expected response of the gatekeeper, without prompt content in any rejected case. For Layer 4, the gatekeeper-side alert on repeated failures fired at the fifth failure, which is the threshold implemented in the function, whereas the three-failure and replay alerts of the design are cloud metric alarms that the local environment does not emulate. On the cloud deployment, a Layer 2 block, a Layer 3 filter, and a configuration change of the agent function all reached the central bus or the alert topic within the one-minute evaluation period of the alarms; the canary case could not be exercised because no run reproduced a canary token. The verification also exposed a defect of the deployment: the policy of the central alert topic authorized the event rules to publish only on behalf of the customer accounts, so the alerts fanned out from the central bus were being denied; the policy was corrected, the table was measured after the correction, and the delivery metrics of the rules before and after it document the cause.

In answer to RQ2, the defense-in-depth claim holds in a specific form, and for the content-path protections: the defensive suffix supplies most of the block rate, but with an effect that depends on the model and is small for the smallest one; Layer 2 adds a model-independent floor of 59.5% at no cost in false positives; and Layer 3 is the component that removes the residual verbatim leakage, measured with the same criterion the layer applies at runtime, so its value lies in what

it had to remove, up to 29.4% of the raw outputs, rather than in the zero itself. Layer 1 and Layer 4 are not factors of this comparison: they were validated functionally, as authentication and integrity controls and as the auditing and response path, respectively. No configuration without Layer 3 achieved zero leakage for all nine models, no configuration without Layer 2 exceeded a mean block rate of 58%, and only the full pipeline combined a mean block rate above 80% with zero leakage across models.

8 Performance and Cost Evaluation

8.1 Latency Overhead

The upper part of Table 9 reports the latency added by each defense layer in the AWS reference implementation. The total overhead at P99 is 68.3 ms, well under the 100–200 ms range that is imperceptible in conversational applications. Cold starts of the container-based function average 1.2 s; provisioned concurrency removes them at approximately \$4.50 per month and per instance, a cost that is not part of the base infrastructure cost reported next.

8.2 Infrastructure Cost

The lower part of Table 9 reports the cost that the defense layers add to a conventional agent deployment on three cloud providers at 50,000 requests per month; Function-as-a-Service (FaaS) denotes the serverless execution of the gatekeeper. The figures are *independent of model inference*, since the defenses operate at the transport and application layers and do not modify inference calls.

The economic value of the architecture is incremental: it adds security controls to a conventional cloud-native agent, at USD 1.37–6.96 per month across the evaluated providers, without dedicated hardware or a specialized inference runtime. This is not a claim of security equivalence with TEEs; it positions the architecture as a low-cost, deployable containment baseline when hardware-backed isolation is unavailable or disproportionate to the threat model.

Table 6: Ablation results per configuration and model: block rate (BR, %), attack success rate (ASR, %), and false-positive rate (FPR, %). Mean over three runs; standard deviations are reported in the repository.

Config.	Metric	Nova Lite	Llama 3.2 1B	Llama 3 8B	Gemma 3 4B	Gemma 4 E4B	Phi-4 Mini	Mistral 7B	Qwen 3.5 4B	Qwen 3.5 9B	Mean
C0	BR	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
	ASR	15.9	3.2	34.1	23.8	2.4	4.0	23.8	9.5	0.0	13.0
	FPR	0.0	0.0	0.0	0.0	13.0	0.0	0.0	0.0	0.0	1.4
C1	BR	93.6	6.3	50.0	50.0	79.4	42.9	71.4	23.8	77.8	55.0
	ASR	0.0	1.6	7.1	0.0	1.6	0.0	19.0	15.9	1.6	5.2
	FPR	13.0	2.9	4.3	4.3	11.6	13.0	4.3	0.0	0.0	6.0
C2	BR	93.7	60.3	85.7	81.0	78.6	76.2	85.7	65.9	84.1	79.0
	ASR	0.8	1.6	0.0	0.0	2.4	0.0	12.7	1.6	0.0	2.1
	FPR	11.6	1.4	4.3	0.0	11.6	13.0	4.3	0.0	0.0	5.2
C3	BR	92.1	8.7	54.8	50.0	73.8	42.9	78.6	40.5	78.6	57.8
	ASR	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
	FPR	4.3	0.0	4.3	2.9	8.7	13.0	8.7	0.0	4.3	5.2
C4	BR	93.7	62.7	85.7	81.0	88.1	73.8	89.7	69.8	84.9	81.0
	ASR	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
	FPR	7.2	0.0	4.3	15.9	11.6	13.0	13.0	0.0	0.0	7.2

Table 7: Complementarity between Layer 2 and Layer 3 (configuration C4[†], Layer 2 in detect-only mode): attack queries by Layer 2 verdict and downstream outcome, summed over the three runs. “L3” includes canary detections; “Passes” gives the number of leaked responses in parentheses.

Model	L2 would block			L2 passes		
	L3	Refuses	Passes	L3	Refuses	Passes
Nova Lite	0	72	0 (0)	2	44	5 (0)
Llama 3.2 1B	0	3	72 (0)	3	0	48 (0)
Llama 3 8B	0	33	42 (0)	0	33	18 (0)
Gemma 3 4B	0	42	33 (0)	0	24	27 (0)
Gemma 4 E4B	3	59	13 (0)	0	36	15 (0)
Phi-4 Mini	0	39	36 (0)	0	21	30 (0)
Mistral 7B	6	57	12 (0)	9	27	15 (0)
Qwen 3.5 4B	11	16	48 (0)	10	11	30 (0)
Qwen 3.5 9B	0	65	10 (0)	0	30	21 (0)

9 Discussion and Limitations

The multi-model evaluation and the ablation study qualify the defense-in-depth claim of the conference version in two ways. First, the effectiveness of the architecture is not uniform across models: the deterministic components, Layer 2 and Layer 3, behave identically for every model, but the prompt-level hardening, which supplies most of the block rate, depends on the instruction-following behavior of the LLM and ranges from negligible (Llama 3.2 1B) to almost complete (Nova Lite). Changing the underlying model therefore changes how early attacks are stopped, but not the leakage, which the output-side layer kept at zero for all nine models. Second, the layers are complementary in function rather than redundant in coverage: Layer 2 stops attacks before inference and never blocked a legitimate query, Layer 3 removes the residual leakage that the model and Layer 2 let through, and the false positives come from the defensive suffix, which several small models over-apply to short legitimate queries. These results argue for treating the defensive suffix as a tunable, model-specific compo-

nent, and for keeping Layer 3 in every deployment regardless of the model.

From a systems perspective, the trust model exchanges hardware-backed guarantees for broader deployability. Its root of trust is a correctly configured cloud IAM, weaker than hardware attestation but available on every provider, and the defense-in-depth design mitigates this weakness: when one layer is compromised, for example through an IAM misconfiguration, the remaining layers continue to enforce protection. The architecture and TEE-based approaches are therefore complementary; recent measurements show TEE overheads below 10% on CPUs and below 7% on GPUs [20], so high-assurance deployments may combine both.

Several limitations remain. Rotating the HMAC signing key requires rebuilding the container image; risk-scoring thresholds may need tuning per deployment; text-processing pipelines that normalize Unicode may strip the zero-width watermark; adversarial fine-tuning on captured prompt–response pairs was not evaluated; and covert signaling mechanisms carry risks of their own [34]. The adversarial evaluation is static: adaptive adversaries that refine their inputs from the observed blocking behavior were not evaluated, and runtime memory access lies outside the threat model of Section 3. The new evaluations add limitations of their own. The leakage oracle detects the verbatim reproduction of 50-character chunks of the prompt and of canary tokens, the same criterion that Layer 3 applies, so a paraphrased disclosure of the instructions does not count as a leak and the outcomes classified as passed without leak must be read with this caveat, in particular for the small models that answered many attacks; an independent oracle applied to the recorded raw outputs is the natural next step. The classification of refusals relies on the fixed refusal sentence and on empty outputs, which undercounts models that decline in their own words and inflated the false-positive rate of one model whose runtime returned empty responses

Table 8: Verification of Layer 1 and Layer 4. The Layer 1 cases and the gatekeeper-side Layer 4 cases were executed against the gatekeeper of the multi-account environment, which runs the production code; the agent-side Layer 4 cases were executed against the two-account cloud deployment and read from the archive of the central event bus and the delivery metrics of its rules. Latencies are measured from the triggering request.

Layer	Case	Adversary	Expected outcome	Observed
L1	Valid signed request	—	HTTP 200, prompt with valid integrity hash	HTTP 200; prompt returned; integrity hash verified
L1	Missing headers; account not in allow-list; timestamp outside the 5-minute window; forged signature; tampered body	$\mathcal{A}_1, \mathcal{A}_2$	HTTP 400 (headers) or 403, no prompt content	As expected in all five cases; body tampering produced its audit event
L1	Replayed nonce	$\mathcal{A}_1, \mathcal{A}_2$	HTTP 403, replay event	First use HTTP 200; replay HTTP 403 with replay audit event
L1	Per-account rate limit exceeded	$\mathcal{A}_2$	HTTP 429, rate-limit event	HTTP 429 with rate-limit audit event (60 requests per minute)
L4	Repeated authentication failures	$\mathcal{A}_1$	Events on central bus; alert raised	Audit events on every failure; gatekeeper alert at the fifth failure (0.01 s); no alert at the third, since the metric alarm with threshold 3 is not emulated locally
L4	Replay attempt	$\mathcal{A}_1, \mathcal{A}_2$	Event on central bus; immediate alert	Replay audit event; no alert locally (metric alarm not emulated)
L4	Five rate-limit rejections within 5 min	$\mathcal{A}_2$	Events on central bus; alert raised	Gatekeeper alert after the fifth rejection (0.01 s)
L4	Layer 2 input block	$\mathcal{A}_2$	Event forwarded within 1 min	PromptInjectionAttempt on the central bus and alert published; 70 s (60-s alarm period)
L4	Layer 3 leakage filter	$\mathcal{A}_2$	Event on central bus	PromptLeakageDetected on the central bus and alert published; 100 s
L4	Canary token in output	$\mathcal{A}_2$	Event on central bus; alert raised	Not exercised: no canary token was reproduced in any run
L4	Customer-side tampering (agent function modified)	$\mathcal{A}_1$	Control-plane event forwarded	Alert published on the central topic 53 s after the change (the rule targets the topic directly)

Table 9: Latency overhead per defense layer over $n = 1,000$ requests (top) and incremental monthly cost of the software-only defense stack over a conventional agent deployment, at 50K requests and excluding model inference (bottom).

Layer	P50	P95	P99
L1: HMAC validation	2.1	3.8	5.2
L2: Risk scoring	8.3	14.7	19.1
L3: Output filter + ZWC	12.5	22.3	31.6
L4: Event forwarding	4.1	8.2	12.4
Total (ms)	27.0	49.0	68.3

Component	Service primitive	AWS	Azure	GCP
Secrets + encryption	Managed secrets + KMS key	\$1.40	\$1.60	\$1.06
Gatekeeper + API	FaaS function + HTTP endpoint	\$0.05	\$0.00	\$0.00
Event bus	Cross-account event routing	\$0.00	\$0.00	\$0.00
Audit storage	Immutable logs (90-day)	\$0.25	\$0.25	\$0.25
Monitoring	Log ingestion + retention	\$0.03	\$0.10	\$0.05
Container registry	Image storage (~100 MB)	\$0.01	\$5.01	\$0.01
Total		\$1.74	\$6.96	\$1.37

to long queries. The corpus is the 42-query corpus of the conference version, so each legitimate query moves the FPR by 4.3 points, and sampling-based decoding was not evaluated. The open-weight models were served at 4-bit or 8-bit quantization on a single GPU, and two of them without the demonstration tools registered, so absolute rates may differ under other serving conditions; the comparison across configurations is unaffected, since each model was evaluated under constant conditions.

From an ethical standpoint, the watermarking mechanism identifies accounts and sessions rather than individual users. Its use should be disclosed transparently and confined to intellectual property protection and

forensic analysis, in line with standard privacy practices.

10 Conclusions and Future Work

This article addressed the protection of AI agent intellectual property in multi-tenant cloud deployments as a cross-domain trust problem and presented a defense-in-depth architecture that reduces prompt exposure and supports output-integrity controls and forensic traceability under the stated trust assumptions, with standard cloud services and without TEEs, at a low latency overhead and an incremental infrastructure cost.

The two experimental contributions answer the questions that a layered design raises. With respect to RQ1, no final response met the verbatim leakage criterion for any of the nine models of six families, while the block rate ranged from 62.7% to 93.7%; the variation is attributable to the model-side hardening, since the input validation layer blocked the same 59.5% of the attacks for every model. With respect to RQ2, the content-path protections contribute different properties: the defensive suffix supplies most of the block rate but with a strongly model-dependent effect, Layer 2 adds a model-independent floor at no cost in false positives, and Layer 3 removes the residual verbatim leakage, which no configuration without it achieved for all models. The complementarity between Layer 2 and Layer 3 was observed directly on the same requests, and the verification of Layers 1 and 4 confirmed the expected behavior of the gatekeeper and of the cross-account alerting on the cloud deployment, where it also exposed and corrected a misconfiguration of the alert topic policy.

Future work will extend the architecture in several directions: optional integration with TEE-based inference for high-assurance deployments, automated rotation of the HMAC signing key without downtime, formal verification of the security properties of the trust chain, and evaluation against adaptive adversaries. Further directions include redundant watermarking that combines ZWC steganography with token-based schemes [30, 32], stateful detection mechanisms for multi-turn interactions, the calibration of the defensive suffix per model family to reduce over-refusal, and leakage oracles that detect paraphrased disclosure.

The practical contribution is therefore economic and operational: a portable, low-cost defense baseline for conventional cloud accounts, while hardware-backed isolation remains appropriate when the threat model includes privileged runtime-memory access.

Authors' contribution

The authors confirm contribution to the paper as follows: DP: Conceptualization, Software, Investigation, Data curation, Validation, Writing – review and editing; JMF: Conceptualization, Methodology, Formal analysis, Writing – original draft, Writing – review and editing. All authors reviewed the results and approved the final version of the manuscript.

Competing interests

The authors have declared that no competing interests exist.

Declaration of Generative AI and AI-assisted Technologies in the Research/Writing Process

During the preparation of this work, the authors used Claude (Anthropic) to assist with the drafting and editing of the manuscript and with the scripts that aggregate the experimental records. After using this tool, the authors reviewed and edited the content as needed and take full responsibility for the content of the publication.

Data Availability

The reference implementation, the reproducibility environment, the corpus, the experimental tooling, and the raw results are available at <https://github.com/dpetrocelli/defense-in-depth-agent-ip>.

References

- [1] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan *et al.*, “Language models are few-shot learners,” *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020.
- [2] R. Bommasani, D. A. Hudson, E. Adeli, R. Altman, S. Arora *et al.*, “On the opportunities and risks of foundation models,” 2021, stanford CRFM report. [Online]. Available: <https://arxiv.org/abs/2108.07258>
- [3] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, “React: Synergizing reasoning and acting in language models,” *arXiv preprint arXiv:2210.03629*, 2023.
- [4] T. Schick, J. Dwivedi-Yu, R. Dessi, R. Raileanu, M. Lomeli, E. Hambro, L. Zettlemoyer, N. Cancedda, and T. Scialom, “Toolformer: Language models can teach themselves to use tools,” *arXiv preprint arXiv:2302.04761*, 2023.
- [5] L. Wang, C. Ma, X. Chen, Q. Yang, Q. Zhang *et al.*, “A survey on large language model based autonomous agents,” *arXiv preprint arXiv:2308.11432*, 2024.
- [6] MarketsandMarkets, “AI agents market—global forecast to 2030,” MarketsandMarkets Research, Tech. Rep., 2024, report ID: TC-9261. Available at <https://www.marketsandmarkets.com/Market-Reports/ai-agents-market-15761548.html>.
- [7] European Union, “General data protection regulation (gdpr),” European Union, Tech. Rep., 2016, regulation (EU) 2016/679.
- [8] Amazon Web Services, “Shared responsibility model,” Amazon Web Services, Tech. Rep., 2023. [Online]. Available: <https://aws.amazon.com/compliance/shared-responsibility-model/>
- [9] A. Shahriar, M. N. Rahman, S. Ahmed, F. Sadeque, and M. R. Parvez, “A survey on agentic security: Applications, threats and defenses,” *arXiv preprint arXiv:2510.06445*, 2025.
- [10] B. Hui, H. Yang, H. Yuan, N. Z. Gong, and L. Lyu, “PLeak: Prompt leaking attacks against large language model applications,” in *Proceedings of the ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2024.
- [11] D. Agarwal, A. R. Fabbri, B. Risher, P. Laban, S. Joty, and C.-S. Wu, “Prompt leakage effect and mitigation strategies for multi-turn LLM applications,” in *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP), Industry Track*, 2024.
- [12] B. C. Das, M. H. Amini, and Y. Wu, “Prompt injection attacks in large language models and AI agent systems: A comprehensive review,” *Information*, vol. 17, no. 1, p. 54, 2026.
- [13] OWASP Foundation, “OWASP top 10 for LLM applications 2025,” Open Web Application Security Project, Tech. Rep., 2025.
- [14] M. A. van Wyk, M. Bekker, X. L. Richards, and K. J. Nixon, “Protect your prompts: Protocols for IP protection in LLM applications,” *arXiv preprint arXiv:2306.06297*, 2023.
- [15] Z. Zhuang, M.-I. Nicolae, H.-P. Wang, and M. Fritz, “ProxyPrompt: Securing system prompts against prompt extraction attacks,” *arXiv preprint arXiv:2505.11459*, 2025.
- [16] B. Cao, C. Li, Y. Cao, Y. Ge, T. Wang, and J. Chen, “You can’t steal nothing: Mitigating prompt leakages in LLMs via system vectors,” in *Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security*, ser. CCS ’25. Association for Computing Machinery, 2025, pp. 4423–4437. [Online]. Available: <https://doi.org/10.1145/3719027.3765124>
- [17] S.-H. Chan, “Encrypted prompt: Securing LLM applications against unauthorized actions,” *arXiv preprint arXiv:2503.23250*, 2025.

- [18] A. Alzahrani, “PromptGuard: A structured framework for injection-resilient language models,” *Scientific Reports*, 2026. [Online]. Available: <https://www.nature.com/articles/s41598-025-31086-y>
- [19] Z. Jiang, Z. Jin, and G. He, “PromptKeeper: Safeguarding system prompts for LLMs,” in *Findings of the Association for Computational Linguistics: EMNLP 2025*. Association for Computational Linguistics, 2025, pp. 2712–2728. [Online]. Available: <https://aclanthology.org/2025.findings-emnlp.147/>
- [20] M. Chrapek, M. Copik, E. Mettaz, and T. Hoefler, “Confidential LLM inference: Performance and cost across CPU and GPU TEEs,” in *Proceedings of the 2025 IEEE International Symposium on Workload Characterization (IISWC)*. IEEE, 2025, pp. 84–98. [Online]. Available: <https://doi.org/10.1109/IISWC66894.2025.00017>
- [21] I. Gim, C. Li, and L. Zhong, “Confidential prompting: Protecting user prompts from cloud LLM providers,” *arXiv preprint arXiv:2409.19134*, 2024.
- [22] T. Bodea, M. Misono, J. Pritzi, P. Sabanic, T. Sommer, H. Unnibhavi, D. Schall, N. Santos, D. Stavrakakis, and P. Bhatotia, “Trusted AI agents in the cloud,” *arXiv preprint arXiv:2512.05951*, 2025.
- [23] J. H. Saltzer and M. D. Schroeder, “The protection of information in computer systems,” *Proceedings of the IEEE*, vol. 63, no. 9, pp. 1278–1308, 1975.
- [24] S. Rose, O. Borchert, S. Mitchell, and S. Connelly, “Zero trust architecture,” *NIST special publication*, vol. 800, no. 207, pp. 1–52, 2020.
- [25] D. Petrocelli and J. M. Fernández, “A defense-in-depth architecture for protecting AI agent intellectual property in multi-tenant cloud environments,” in *Cloud Computing, Big Data & Emerging Topics (JCC-BD&ET 2026)*, ser. Communications in Computer and Information Science (CCIS). Springer, 2026, in press.
- [26] K.-H. Hung, C.-Y. Ko, A. Rawat, I.-H. Chung, W. H. Hsu, and P.-Y. Chen, “Attention tracker: Detecting prompt injection attacks in LLMs,” in *Findings of ACL: NAACL 2025*, 2025, pp. 2309–2322.
- [27] S. Thornton, “TRYLOCK: Defense-in-depth against LLM jailbreaks via layered preference and representation engineering,” *arXiv preprint arXiv:2601.03300*, 2026.
- [28] D. Schwarz, “Countermind: A multi-layered security architecture for large language models,” *arXiv preprint arXiv:2510.11837*, 2025.
- [29] C. Pathade, V. Dhimam, S. Ahmad, and I. Lareb, “Serverless AI security: Attack surface analysis and runtime protection mechanisms for FaaS-based machine learning,” *arXiv preprint arXiv:2601.11664*, 2026.
- [30] J. Kirchenbauer, J. Geiping, Y. Wen, J. Katz, I. Miers, and T. Goldstein, “A watermark for large language models,” in *Proceedings of the 40th International Conference on Machine Learning (ICML)*, 2023.
- [31] G. K. R. Lau, X. Niu, H. Dao, J. Chen, C.-S. Foo, and B. K. H. Low, “Waterfall: Framework for robust and scalable text watermarking and provenance for LLMs,” in *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2024, pp. 20 432–20 466.
- [32] S. Dathathri, A. See, S. Ghaisas, P.-S. Huang, R. McAdam, J. Welbl *et al.*, “Scalable watermarking for identifying large language model outputs,” *Nature*, vol. 634, pp. 818–823, 2024.
- [33] H. M. Bashir, Q. Li, and J. Hou, “A high capacity text steganography utilizing unicode zero-width characters,” in *2020 IEEE International Conferences on Internet of Things (iThings), GreenCom, CPSCom, SmartData and Cybermatics*. IEEE, 2020, pp. 668–675.
- [34] Y. Mathew, O. Matthews, R. McCarthy, J. Velja, C. Schroeder de Witt, D. Cope, and N. Schoots, “Hidden in plain text: Emergence and mitigation of steganographic collusion in LLMs,” in *Proceedings of IJCNLP-AACL*, 2025.
- [35] National Institute of Standards and Technology, “Adversarial machine learning: A taxonomy and terminology of attacks and mitigations,” NIST, Tech. Rep. AI 100-2e, 2025.

Citation: D. Petrocelli and J. M. Fernández. *Layered Prompt Protection for AI Agents in Customer-Controlled Cloud Accounts: Model Robustness and Defense-Layer Contribution*. Journal of Computer Science & Technology, vol. XX, no. X, pp. X–X, 2026.

DOI: to be assigned at the copyediting stage.

Received: to be completed. **Accepted:** to be completed.

Copyright: This article is distributed under the terms of the Creative Commons License CC-BY-NC-SA.